

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

Register Number	RAVURU PRANATHI
Name	192525076

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

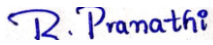
Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I **R.Pranathi(192525076)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

ANS:

A media platform stores a large amount of data such as videos, images, audio files and user-generated content. Since the amount of data is continuously increasing, HDFS can be used to provide distributed and reliable storage. The main constraints in this problem are that data availability must be above 99.9%, replication factor cannot be more than 3, and storage growth should remain cost efficient.

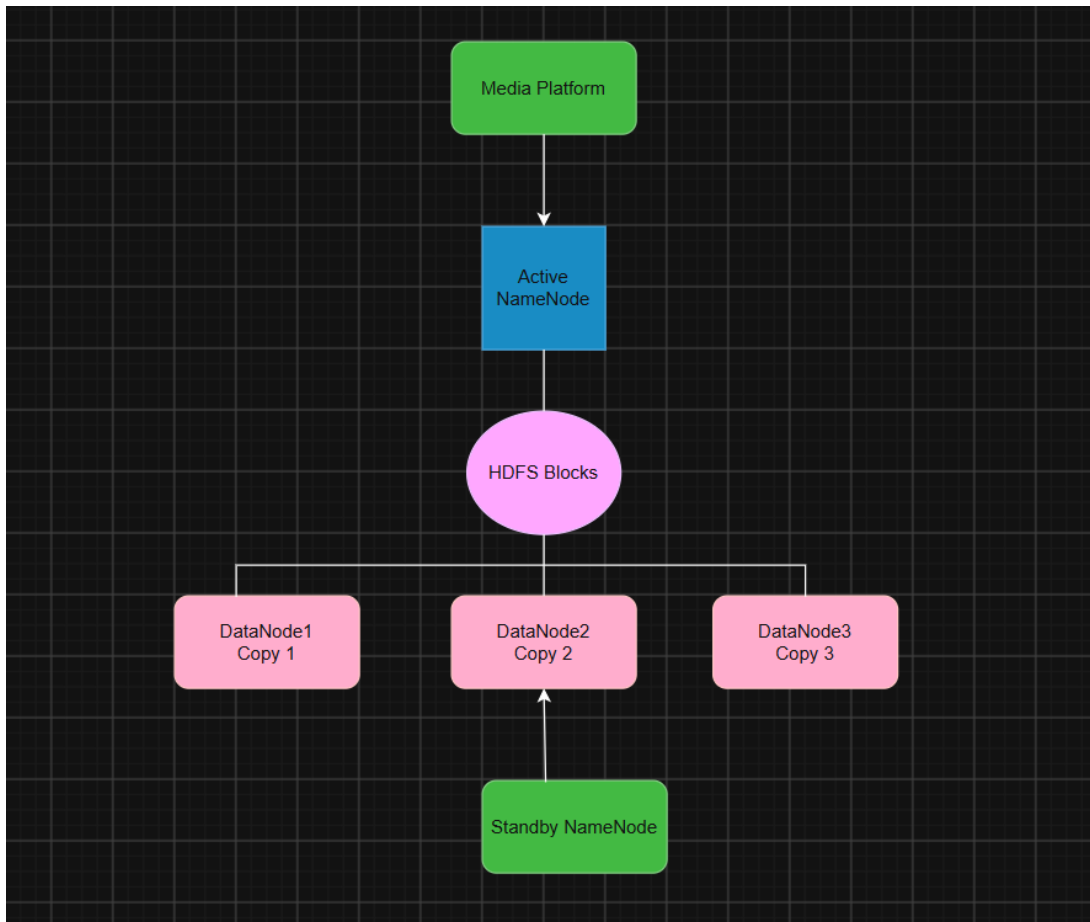
The HDFS cluster can have a **NameNode** to manage file information and multiple DataNodes to store the actual media files. The media files are divided into blocks and these blocks are distributed across different DataNodes. This allows the system to store very large files without depending on one server.

For high availability, the HDFS cluster can use an Active NameNode and Standby NameNode. The Standby NameNode can take over if the Active NameNode fails. This prevents the NameNode from becoming a single point of failure and helps the system maintain availability above 99.9%.

The replication factor should be set to **3**, which is the maximum allowed by the constraint. Each HDFS block will have three copies stored on different DataNodes. If one DataNode fails, another copy of the

block is available. HDFS also places replicas on different racks when possible. This protects the data even if one rack has a failure.

The design can be shown as:



For cost-efficient storage growth, the platform can start with a smaller number of DataNodes and add more nodes when the amount of media data increases. This is called **horizontal scaling**. It is more economical than replacing the entire storage system with a larger server.

The company should also monitor disk usage regularly. When storage reaches a predefined limit, new DataNodes can be added. This allows the system to grow according to actual requirements instead of purchasing large storage capacity in advance.

Media files can also be compressed before storing them. Older or less frequently accessed media can be moved to cheaper storage when suitable. This reduces the amount of expensive storage required.

The replication factor of 3 provides a good balance between reliability and storage cost. A replication factor of 1 would save storage but would provide poor fault tolerance. A factor higher than 3 would improve redundancy but violates the given constraint and increases storage cost. Therefore, **replication factor 3 is the most suitable choice**.

For example, if a video is divided into three HDFS blocks, each block can have three copies distributed across different DataNodes. If one DataNode fails, users can still access the video from another copy.

HDFS also continuously monitors DataNodes. If a DataNode fails, HDFS can identify the failure and use the available replica. A replacement replica can later be created on another healthy DataNode while keeping the replication factor within the allowed limit.

Therefore, the proposed solution uses HDFS with Active/Standby NameNodes, a maximum replication factor of 3, multiple DataNodes, rack-aware replica placement, compression and horizontal scaling. The Active/Standby NameNodes and three replicas help maintain **more than 99.9% availability**, while adding DataNodes only when required and using compression helps keep storage growth cost efficient.

In conclusion, this HDFS design satisfies all three constraints: high availability is achieved through NameNode failover and replication, the replication factor remains at the allowed maximum of **3**, and cost is controlled through scalable storage, compression and gradual addition of DataNodes.

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

ANS:

A company needs to process a large amount of log data using MapReduce. The main constraint is that the complete job must finish within a **20-minute batch window**, and the number of cluster nodes is limited. Therefore, the MapReduce job should be designed to use the available nodes efficiently and reduce unnecessary processing.

First, the log files are stored in **HDFS**. HDFS divides the log files into blocks and distributes them across the available DataNodes. MapReduce can process these blocks in parallel.

The **Mapper** reads each log record and extracts useful information such as IP address, URL, status code and number of requests. For example, if the requirement is to count website requests, the mapper can produce:

/page1 → 1

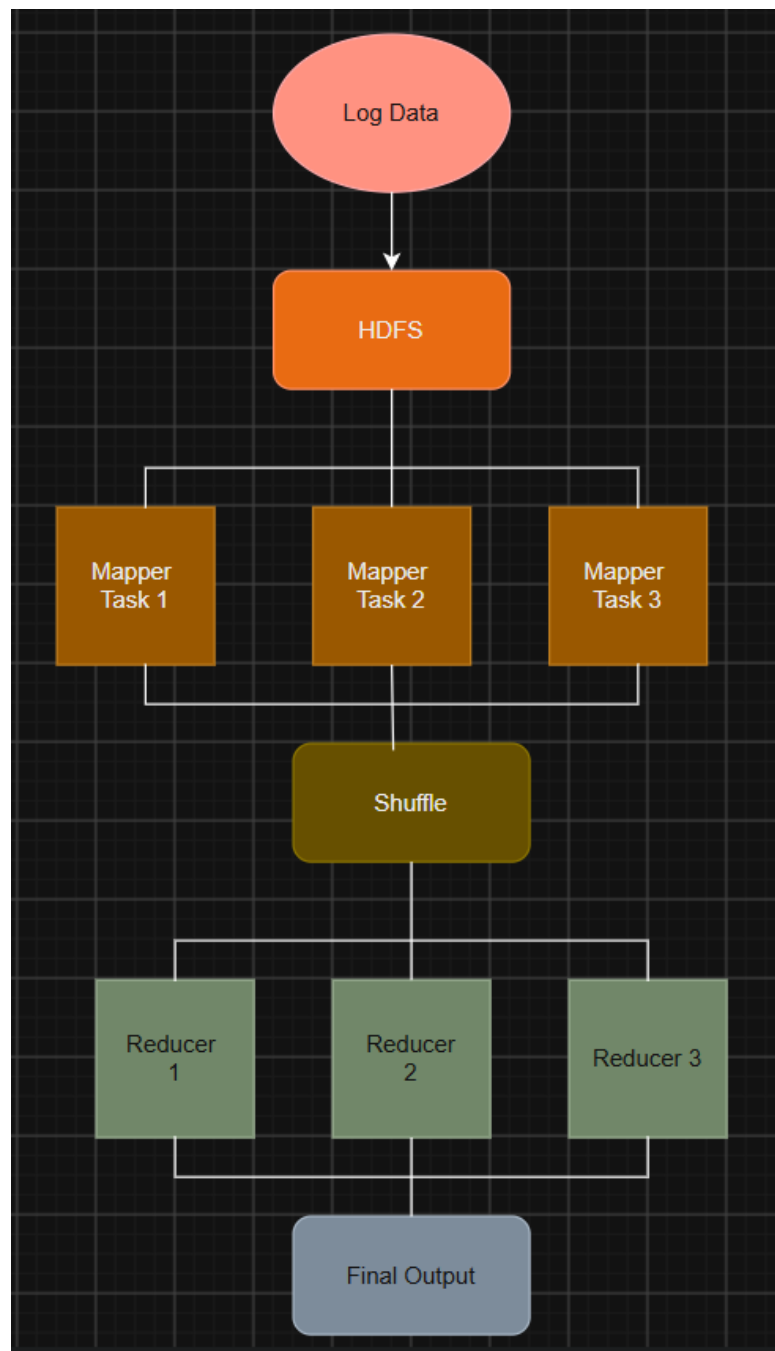
/page2 → 1

/page1 → 1

The **Reducer** receives all values belonging to the same key and adds them together. The final output may be:

/page1 → 2

/page2 → 1



Since the cluster has limited nodes, the number of mapper and reducer tasks should be controlled according to the available CPU and memory. Too many tasks can increase overhead and slow down the job.

The input data should be divided into suitable HDFS blocks so that multiple mapper tasks can run in parallel. Data locality should be used whenever possible, meaning the mapper should run on the node where the required data is already stored. This reduces network data transfer and improves processing speed.

The mapper should perform simple filtering and transformation before sending data to the reducer. Unnecessary records should be removed early. This reduces the amount of data transferred during the shuffle stage.

The number of reducers should also be selected carefully. With limited nodes, a small number of properly sized reducers can provide better performance than creating too many reducers.

To meet the 20-minute constraint, the job should be tested using sample log data and the execution time should be monitored. If the job is taking too long, mapper and reducer tasks can be adjusted, unnecessary processing can be removed and input data can be partitioned properly.

For example, if 60 GB of logs are stored across three available processing nodes, the data can be divided among the nodes so that each mapper processes approximately 20 GB. The mapper tasks run in parallel, followed by shuffle and reducer tasks.

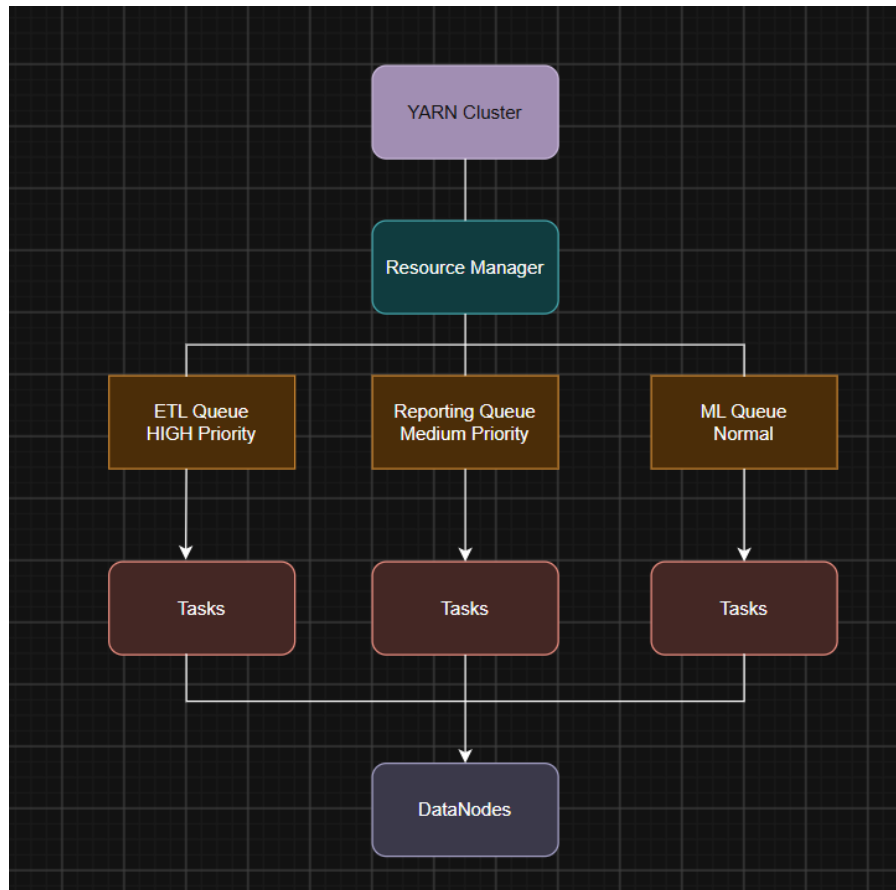
Therefore, the proposed MapReduce design uses HDFS for distributed storage, parallel mapper tasks, shuffle and partitioning, and a limited number of reducer tasks. Data locality and early filtering reduce network traffic and processing time. With proper task distribution and resource allocation, the log-processing job can be designed to complete within the required **20-minute batch window**.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

ANS:

A YARN-managed cluster is used to run three types of jobs: ETL, reporting and machine learning. The main constraints are fairness between jobs, higher priority for ETL jobs, and recovery when a node fails. Therefore, the scheduler should allocate cluster resources carefully so that one type of job does not use all the available resources.

I would use a Capacity Scheduler or a similar queue-based scheduling approach. Separate queues can be created for ETL, reporting and machine learning jobs.



The **ETL queue should receive the highest priority** because ETL jobs prepare and transform data that may be required by reporting and machine learning jobs. However, ETL should not be allowed to use all cluster resources.

The reporting queue can receive a medium priority. Reporting jobs should get a reasonable share of resources so that business reports are generated on time.

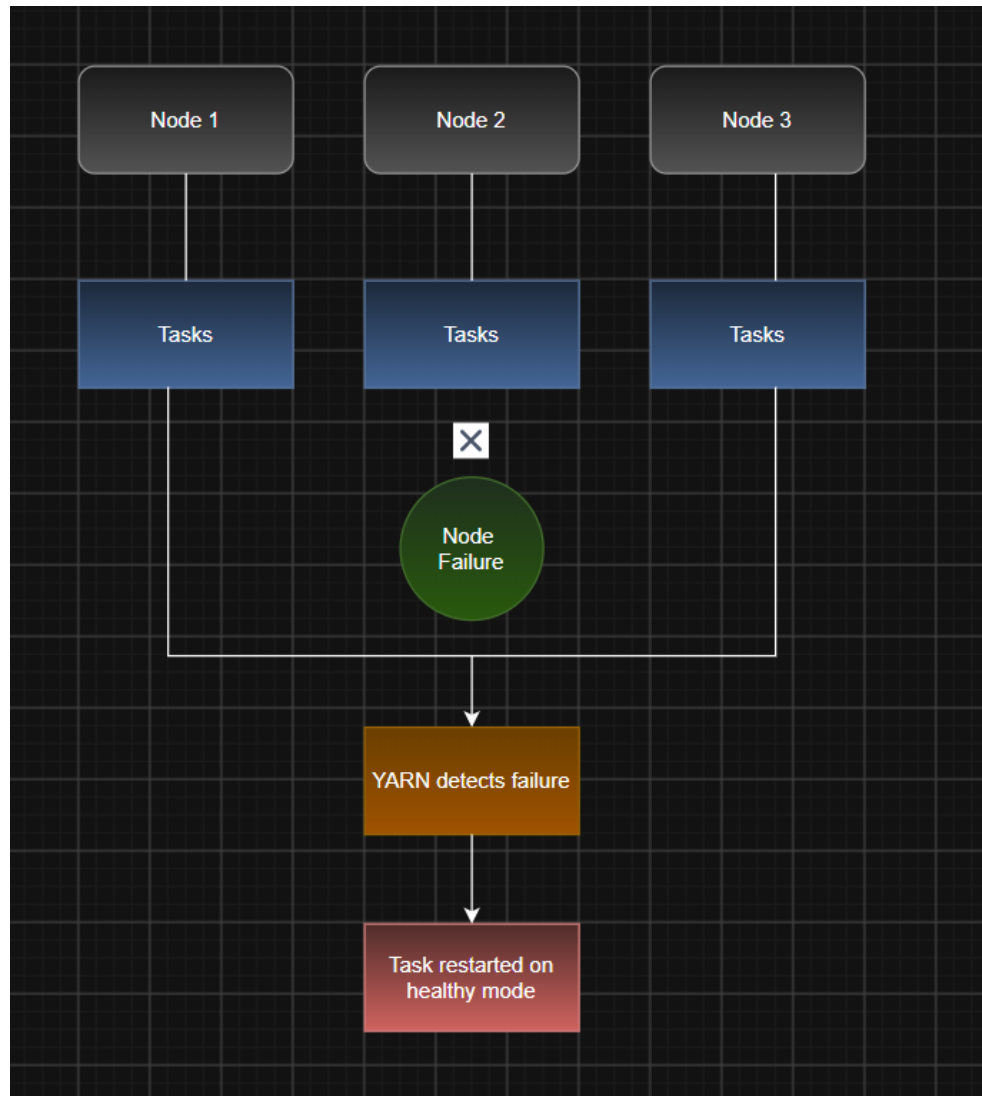
Machine learning jobs can have normal priority. Since ML jobs may require large amounts of CPU and memory, a resource limit can be applied to prevent them from affecting ETL and reporting jobs.

Fairness can be maintained by giving each queue a minimum percentage of cluster resources. For example, the cluster could allocate more resources to ETL, while still reserving resources for reporting and ML.

If an ETL job requires more resources and other queues are not using their allocated resources, the scheduler can temporarily give the unused resources to ETL. When reporting or ML jobs require resources, they can receive their fair share again.

YARN's **ResourceManager** manages the overall cluster resources, while **NodeManagers** run and monitor tasks on individual nodes.

Node failure must also be handled. If a DataNode or NodeManager fails while running a task, YARN can detect the failure through the NodeManager and ResourceManager. The failed task can be restarted on another healthy node.



This recovery mechanism prevents the complete job from failing because of one node failure.

The scheduler should also monitor CPU and memory usage. Resource limits can be applied to individual queues so that a large machine learning job does not consume all available resources.

For example, if an ETL job, a reporting job and a machine learning job are running at the same time, the scheduler gives ETL higher priority but still keeps resources available for reporting and ML. If one worker node fails, its running task is restarted on another available node.

Therefore, the proposed YARN policy provides higher priority to ETL, fair resource sharing for reporting and ML, and automatic task recovery after node failure. This gives a good balance between performance, fairness and reliability while making effective use of the limited cluster resources.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

ANS:

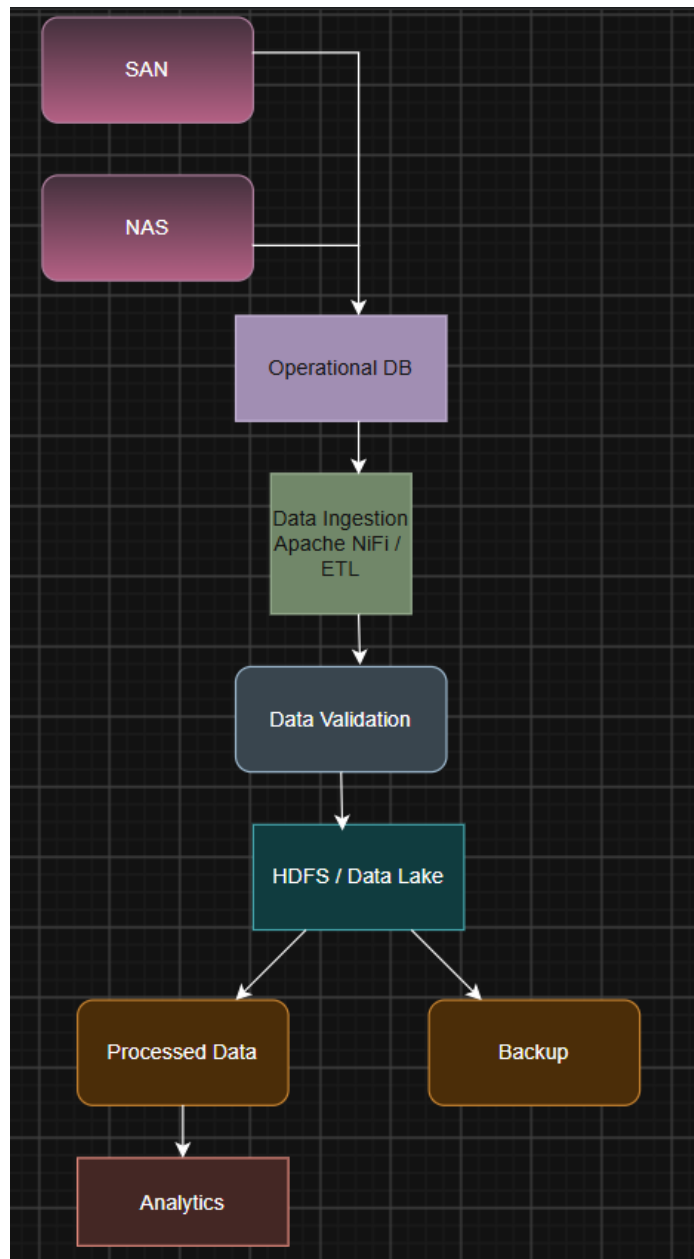
An enterprise needs to collect data from SAN, NAS and operational databases and move it into a big data platform. The main constraints are reliable backup and minimum duplication of data. Therefore, the architecture should collect data efficiently, avoid unnecessary copies and maintain a reliable backup.

SAN and NAS contain files and enterprise data, while operational databases contain structured information such as customer, sales and transaction records. An ingestion layer can be used to collect data from all these sources.

For operational databases, an ETL/ELT tool such as Apache NiFi can be used. NiFi can connect to different data sources and transfer data to the big data platform. For large database tables, incremental or change-based ingestion should be used instead of copying the complete database every time.

For SAN and NAS, NiFi or suitable file ingestion tools can monitor folders and transfer only new or changed files. This helps reduce unnecessary duplication.

The data can then be stored in HDFS or a cloud-based data lake. HDFS provides distributed storage and can replicate data across different DataNodes. This provides backup reliability even if one storage node fails.



To maintain **minimal duplication**, the ingestion system should use incremental loading. Only new or modified records should be transferred. Unique IDs such as customer ID, transaction ID and file name can be used to identify duplicate data.

Data validation should also be performed before storing the information. If the same file or record already exists, the system should avoid storing another unnecessary copy.

For backup reliability, HDFS replication can be used. Important data can have multiple replicas across different DataNodes. However, replication should be controlled because too many copies increase storage costs.

A backup policy should also be maintained for critical data. Important datasets can be backed up to another storage location so that they can be recovered if the main system fails.

For example, if a customer database contains one million records and only 10,000 records are newly added, the ingestion system should transfer only those new records instead of copying all one million records again. This saves storage and network bandwidth.

Therefore, the proposed architecture uses Apache NiFi/ETL for ingestion, incremental loading for minimum duplication, HDFS/Data Lake for distributed storage, validation for data quality and controlled replication for backup reliability. This design provides reliable data ingestion while avoiding unnecessary copies and storage costs.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

ANS:

An enterprise needs a complete Hadoop-based system that can collect, store and process large amounts of data. The main constraints are secure data movement, high availability and integration with Apache NiFi or ETL pipelines. Therefore, the system should be designed so that data is transferred securely and the platform continues working even if one component fails.

First, Apache NiFi can be used as the data ingestion layer. It can collect data from databases, files, applications and other sources and transfer it into the Hadoop platform. NiFi also provides data flow monitoring and control.

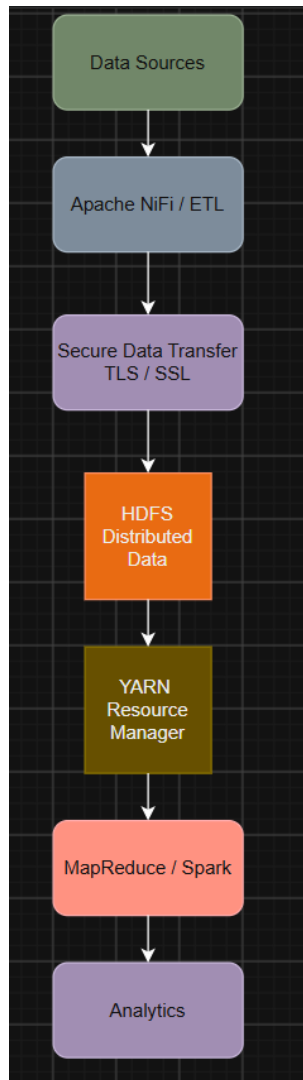
For secure data movement, encryption such as TLS/SSL can be used when data moves between the source systems, NiFi and Hadoop. Authentication should also be enabled so that only authorized systems can send or receive data.

The data received by NiFi can be stored in HDFS. HDFS divides large files into blocks and distributes them across different DataNodes. A replication factor of 3 can be used so that copies of data are available on different nodes.

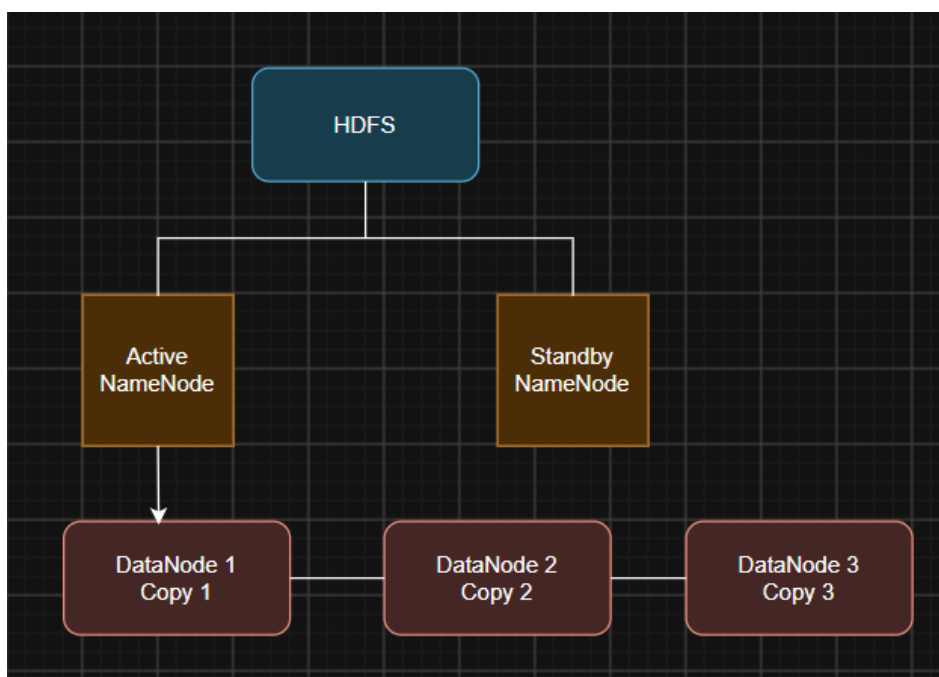
The Hadoop ecosystem can use YARN for resource management. YARN allocates CPU and memory to different applications and allows multiple jobs to run on the same cluster.

MapReduce or Apache Spark can then be used for processing the data stored in HDFS. MapReduce is suitable for batch processing, while Spark can be used for faster data processing and analytics.

For high availability, the Hadoop cluster can use an Active and Standby NameNode. If the Active NameNode fails, the Standby NameNode can take over. Data is also replicated across multiple DataNodes, so failure of one DataNode does not cause data loss.



For **high availability**, the storage layer can be designed as:



NiFi can also be configured to handle failures in the data flow. If a transfer fails, the data can be retried instead of being lost. This improves the reliability of the ingestion process.

Security can also be strengthened using authentication, authorization and audit logs. Access to HDFS and other Hadoop services should be given only to authorized users and applications.

For example, an enterprise can use NiFi to collect sales data from databases, securely transfer it to HDFS and then use YARN to allocate resources for MapReduce or Spark jobs. If one DataNode fails, another replica can provide the required data. If the Active NameNode fails, the Standby NameNode can continue the service.

Therefore, the proposed solution uses Apache NiFi/ETL for data ingestion, TLS/SSL for secure data movement, HDFS for distributed storage, YARN for resource management and MapReduce/Spark for processing. NameNode high availability and data replication improve reliability, while NiFi provides controlled and monitored integration with the Hadoop ecosystem.

This end-to-end architecture satisfies all three constraints by providing **secure data movement, high** availability through replication and failover, and smooth integration through Apache NiFi/ETL pipelines.